TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education
HCMUTE

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

React.js

THS. NGUYỄN HỮU TRUNG

Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Nội dung

- React.js là gì?
- Cài đặt React.js
- React.js render HTML
- React.js JSX
- React.js Component
- React.js props
- React.js props xử lý data

React.js là gì ?

- React.js là một thư viện Javascript để xây dựng giao diện người dùng, được phát triển ban đầu từ Facebook và đóng góp bởi cộng đồng lập trình viên trên thế giới.
- React.js có thể sử dụng để viết phần nền cho single-page hoặc ứng dụng di động, giúp trang lấy và xử lý dữ liệu được tối ưu hơn các phương pháp khác.
- React.js thích hợp với các ứng dụng lớn, khả năng mở rộng ở tương lai.
- Điểm mạnh của React.js dễ thấy nhất có lẽ là phần Component:
  - React.js chia nhỏ các phần của trang thành từng phần riêng biệt để xử lý, gọi là component giúp dễ quản lý, dễ sử dụng ở nhiều nơi
  - Mỗi khi dữ liệu được cập nhật mới, thay vì thay đổi nguyên trang, thì React.js sẽ giúp thay đổi chỉ component liên quan, việc này sẽ tối ưu rất nhiều thời gian làm mới dữ liệu.
  - Hầu hết các component thường dùng đã được phát triển và chia sẻ, chỉ cần cài đặt và sử dụng.

Cài đặt React.js

- Cài đặt trực tiếp React.js từ tag **<script>**

index.html 1 ×

index.html > html > body

```html
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8" />
    <title>Hello React</title>
    <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>

<body>
    <div id="root"></div>

    <script type="text/babel">
        ReactDOM.render(
        <h1>Hello, React!</h1>,
        document.getElementById('root')
      );
    </script>
</body>

</html>
```

** ,** React sẽ load nội dung tới **id** này.

react.development.js và **react-dom.development.js**, là 2 file thư viện của React.

babel.min.js, cần thiết cho cấu trúc JSX (Javascript XML - viết cấu trúc XML trong Javascript, như trên là cấu trúc <h1>Hello, world!</h1>), nếu bạn viết React mà không dùng cấu trúc JSX thì không cần thêm <script> này

```html
<script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
```

Cài đặt React.js

- Cài đặt React.js bằng lệnh
  - Để cài đặt một project React hoàn chỉnh (liên kết file, sử dụng component riêng biệt, tạo thư viện tùy ý, ...) ta thực hiện các bước sau:
    - Bước 1 - Cài đặt môi trường Node.js
    - Bước 2 - Cài đặt bộ cài app React
    - Bước 3 - Tạo project React.js
    - Bước 4 - Thực thi lệnh chạy

Cài đặt React.js

- Cài đặt React.js bằng lệnh
  - Bước 1 - Cài đặt môi trường Node.js: Xem lại bài cài đặt chi tiết Node.js.
  - Bước 2: Cài đặt React, vào **command prompt** hoặc **Terminal của VSCode** gõ:

npm install -g create-react-app

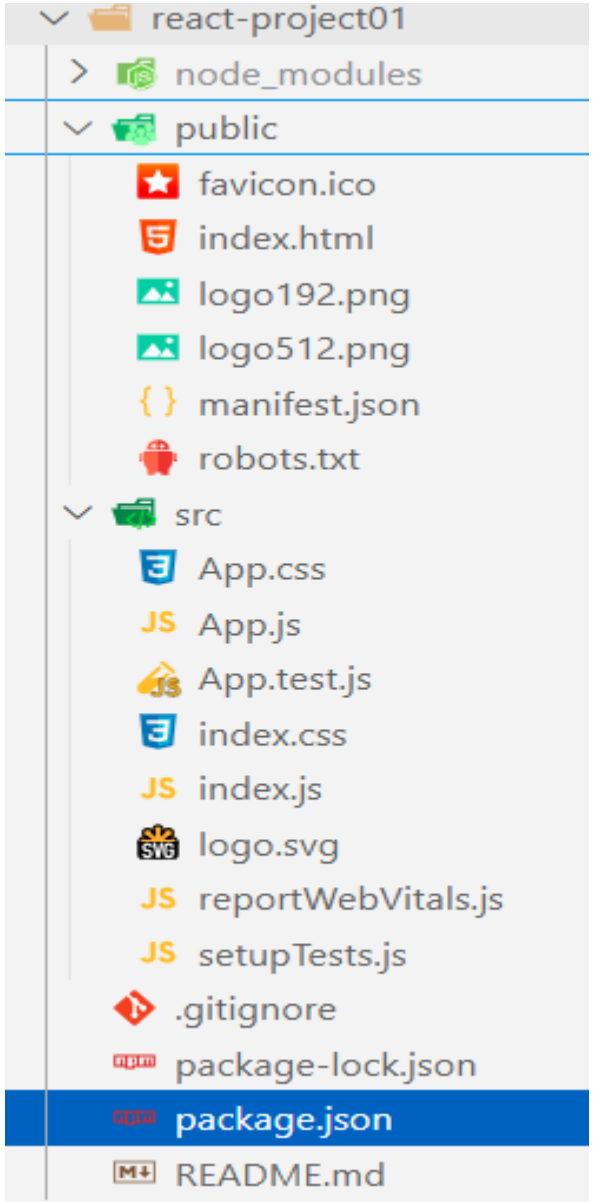
Vào thư mục cần tạo project: gõ **cmd** -> rồi gõ lệnh sau: create-react-app **react-project01**

Sau khi khởi tạo project thành công, ta gõ: **cd react-project01** -> rồi gõ **npm start** để khởi động project.

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL CONSOLE

```
  npm audit fix --force

Run `npm audit` for details.

Created git commit.

Success! Created react-project01 at E:\Nam2025\CongNghePhanMemMoi\project_Fulltask\reactjs-slide\react-project01
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can’t go back!

We suggest that you begin by typing:

  cd react-project01
  npm start

Happy hacking!
PS E:\Nam2025\CongNghePhanMemMoi\project_Fulltask\reactjs-slide>
```

Cài đặt React.js



node_modules: chứa nội dung cài đặt, tất cả cài đặt sẽ được lưu tại đây, chúng ta không thao tác trong thư mục này.

public: chứa tất cả file output, là các file sẽ tương tác trực tiếp với trình duyệt như: HTML, image, ...

src: chứa tất cả các file input, đây là các file mà chúng ta sẽ code nội dung, thao tác phần lớn ở những file này, gồm các file Javascript, CSS,...

.gitignore: file này quy định những file/folder nào không cần đẩy lên remote server (Github/Gitlab...)

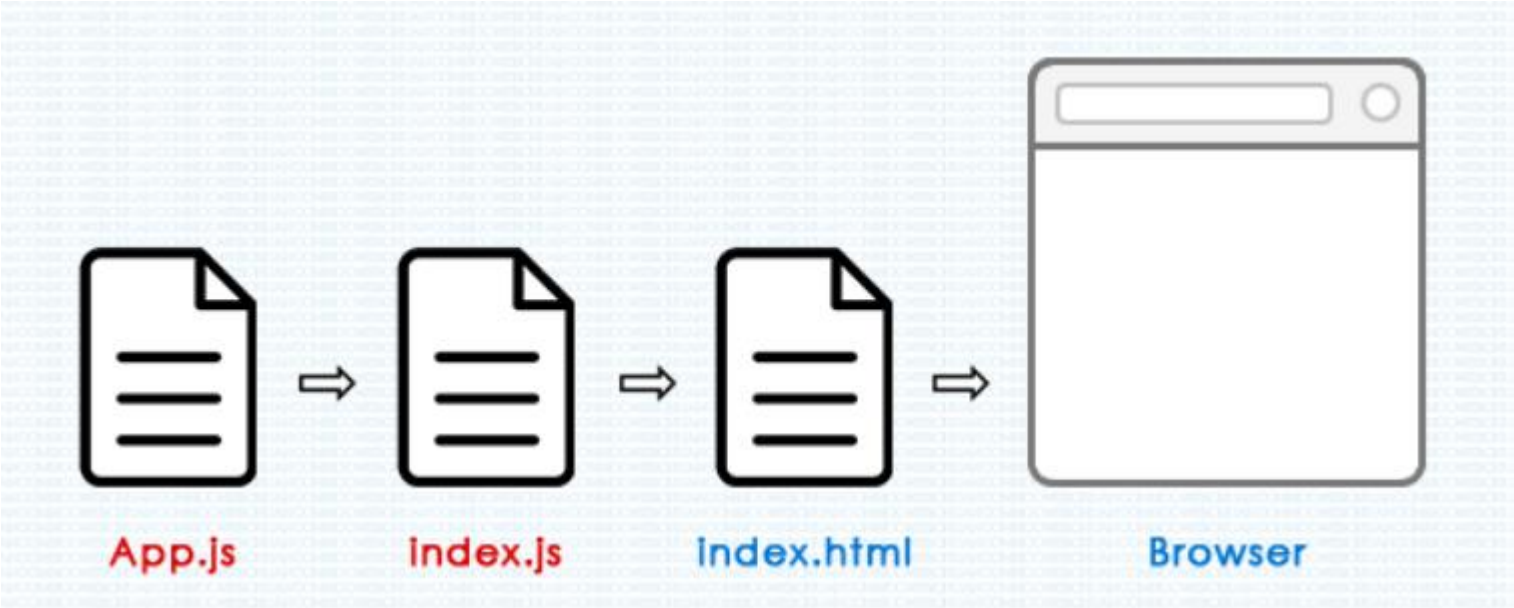
package.json ×

react-project01 > package.json > ...

```
1  {
2    "name": "react-project01",
3    "version": "0.1.0",
4    "private": true,
5    "dependencies": {
6      "@testing-library/jest-dom": "^5.17.0",
7      "@testing-library/react": "^13.4.0",
8      "@testing-library/user-event": "^13.5.0",
9      "react": "^18.3.1",
10     "react-dom": "^18.3.1",
11     "react-scripts": "5.0.1",
12     "web-vitals": "^2.1.4"
13   },
     Debug
14   "scripts": {
15     "start": "react-scripts start",
16     "build": "react-scripts build",
17     "test": "react-scripts test",
18     "eject": "react-scripts eject"
19   },
20   "eslintConfig": {
21     "extends": [
22       "react-app",
23       "react-app/jest"
24     ]
25   },
26   "browserslist": {
27     "production": [
28       ">0.2%",
29       "not dead",
30       "not op_mini all"
31     ],
32     "development": [
33       "last 1 chrome version",
34       "last 1 firefox version",
35       "last 1 safari version"
36     ]
```

React.js render HTML

- Render là việc hiển thị nội dung lên trình duyệt. Nội dung có thể được viết từ HTML, Javascript hay PHP, Java, C#, ... Kết quả cuối cùng là hiển thị nội dung đó trên trình duyệt cho người dùng sử dụng.
- Với React.js thì nội dung layout bạn viết không phải nằm ở trang HTML, mà được viết bên trong file Javascript, file HTML chỉ là cầu nối giúp nội dung Javascript "liên kết" với trình duyệt.



React.js render HTML

- Cách React render nội dung HTML ra trình duyệt: Để hiểu rõ hơn về render trong React, ta xem xét nội dung 3 file:

index.html 1

react-project01 > public > index.html > ...

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>React App</title>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

index.js

react-project01 > src > index.js > ...

```javascript
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
reportWebVitals();
```

App.js

react-project01 > src > App.js > default

```javascript
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Hello Nguyễn Hữu Trung
        </p>
      </header>
    </div>
  );
}

export default App;
```

React.js JSX

- JSX (Javascript XML), là cấu trúc XML được viết bên trong cấu trúc Javascript (HTML cũng được viết theo cấu trúc XML nhé), do đó có thể hiểu đơn giản hơn JXS giống như cách viết HTML bên trong cấu trúc Javascript.
- Vậy tại sao cần JSX? các bạn thử xem 2 cấu trúc sau nhé

Cấu trúc React	Cấu trúc React viết theo JSX

`const element = React.createElement(`
`  'h1',`
`  {className: 'greeting'},`
`  'Hello, world!'`
`);`
	`const element = (`
	`  <h1 className="greeting">Hello, world!</h1>`
	`);`

- Để render thẻ `<h1>` ta thấy rõ ràng cách viết JSX ngắn gọn hơn, và nó gần giống với cấu trúc HTML quen thuộc.
- Đây chỉ mới có 1 thẻ `<h1>`, nếu nội dung là một file HTML hoàn chỉnh thì cấu trúc bên trái sẽ rất phức tạp.
- Ngoài ra JSX chống tấn công kiểu injection, do mặc định React DOM sẽ loại bỏ những ký tự đặc biệt trong bất kì giá trị nào được nhúng vào JSX trước khi được render, việc này giúp ngăn chặn phương thức tấn công XSS (cross site scripting).
- Đó là lý do chúng ta nên viết React theo cấu trúc JSX.

React.js JSX

- JSX được viết ở đâu?

JS App.js M ×

react-project01 > src > JS App.js > App

```
1   import logo from './logo.svg';
2   import './App.css';
3
4   function App() {
5     return (
6       <div className="App">
7         <header className="App-header">
8           <img src={logo} className="App-logo" alt="logo" />
9           <p>
10            Hello Nguyễn Hữu Trung
11          </p>
12        </header>
13      </div>
14    );
15  }
16  export default App;
```

Khác biệt giữa HTML và JSX

- Cấu trúc JSX tương đồng với cấu trúc HTML, tuy nhiên có một số điểm khác biệt ta cần chú ý:

Mô tả	Cấu trúc HTML	Cấu trúc JSX
Tên Class	<tag class="">	<tag className="">
Thuộc tính `value` <input />	<input value="" />	<input defaultValue="" />
Thuộc tính `for` của <label>	<label for="">	<label htmlFor="">
Giá trị của <select><option>	<option value="">	<option value={}>
Style trực tiếp bên trong tag	<tag style="width: 10%">	<tag style={{ width: '10%' }}>
Event	<tag onclick="functionName()">	<tag onClick={functionName}>
Khi viết về giá trị được gọi		<img src={path}> Hello {name}!

- Hiện tại có nhiều trang hỗ trợ việc chuyển đổi nội dung HTML sang JSX, các bạn có thể tham khảo: HTML to JXS. Tùy vào trường hợp mà các bạn code từng phần hoặc dùng công cụ chuyển đổi cho phù hợp nhé.

React.js Component

- React chia nhỏ các phần của trang thành từng phần riêng biệt, gọi là component, ví dụ như các phần: header, footer, sidebar, navigation, itemList, ...
- Tính chất của component giống như một hàm (function) Javascript, có thể tái sử dụng ở nhiều nơi khác nhau.
- Cách viết một component
  - Viết dưới dạng function (hoặc Class - nói ở bài sau), và function luôn được return.
  - Bên trong return luôn tồn tại tag bao ngoài tất cả (tag wrap).
  - Sử dụng thư viện React.DOM để render một component
  - Function render phải có cấu trúc XML, viết dưới dạng <Tên /> hoặc <Tên></Tên>
- Chú ý là chúng ta tạm thời sẽ sử dụng cách cài đặt trực tiếp React.js để hiểu các khái niệm cũng như tính năng của React nhé, sau này tới bài thao tác Project chúng ta sẽ thực hành Project React cài đặt bằng lệnh sau.

React.js Component

index.html 1 ×

index.html > html > body > script

```html
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8" />
    <title>Hello React</title>
    <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>

<body>
    <div id="member"></div>

    <script type="text/babel">
        function Member(){
        return(
          <div className="member">
            <h2>Nguyễn Hữu Trung</h2>
            <p>Tuổi: 42</p>
          </div>
        )
      };
      ReactDOM.render(
        <Member />,
        document.getElementById('member')
      );
    </script>
</body>

</html>
```

- **Function Member()** đóng vai trò là một component, nhiệm vụ là tạo một đối tượng member với tên và tuổi.
- **ReactDOM.render** giữ vai trò render nội dung ra trình duyệt, dựa vào **id="member".**

React.js Component

- Cách tạo và include component: Giả sử ta có nhiều component khác nhau trong một trang, mỗi component có một nhiệm vụ riêng, Ví dụ:
  - Component avatar dành để xử lý hình ảnh member.
  - Component memberInfo dành để hiển thị thông tin member như tên, tuổi.
  - Component comment dành để quản ý bình luận của member.
- Nhiệm vụ của ta là làm sao viết kết hợp các component này lại để hiển thị thông tin cho member, ta làm như sau:

React.js Component


```
<div id="member"></div>
<script type="text/babel">
    function Avatar(){
    return(
      <div className="avatar">
        <img src="img_avatar.jpg" alt="" />
      </div>
    )
  };
  function MemberInfo(){
    return(
      <div className="info">
        <h2>Nguyễn Hữu Trung</h2>
        <p>Tuổi: 42</p>
      </div>
    )
  };
  function Comment(){
    return(
      <div className="comment">
        Giảng viên khoa CNTT - HCMUTE.
      </div>
    )
  };
  function Member(){
    return(
      <div className="member">
        <Avatar />
        <MemberInfo />
        <Comment />
      </div>
    )
  };
```

```
  ReactDOM.render(
    <Member />,
    document.getElementById('member')
  );
</script>
```

React.js Props

- Component thực chất là một function Javascript (hoặc class), phương thức truyền giá trị giữa các function này được gọi là **props**, chúng được React xử lý và trả kết quả hiển thị trên trình duyệt.
- Cách viết một React.js Props: Thay vì viết cố định tên và tuổi bên trong component Member, thì ta có thể viết như sau:

```
<div id="member"></div>
<script type="text/babel">
    function Member(props){
    return(
      <div className="member">
        <h2>{ props.name }</h2>
        <p>Tuổi: { props.age }</p>
      </div>
    )
  };

  ReactDOM.render(
    <Member name="Nguyễn Hữu Trung" age="42" />,
    document.getElementById('member')
  );
</script>
```

Ở component **Member** ta đã sử dụng **props**, giống như cách báo cho hệ thống React biết nơi nào xuất hiện props thì nơi đó sẽ được liên kết với các biến (name, age), khi render thì các tham số (Nguyễn Văn A, 25) sẽ được truyền vào các biến này

React.js Props

- Tái sử dụng Component trong cùng khu vực (danh sách, nhóm, ...): Giả sử ta cần render thêm một member với thông tin “Hữu Trung", “40" tuổi, thay vì viết thêm một component mới, thì ta có thể sử dụng lại component đã có như sau:

```
<div id="member"></div>
<script type="text/babel">
    function Member(props){
    return(
      <div className="member">
        <h2>{ props.name }</h2>
        <p>Tuổi: { props.age }</p>
      </div>
    )
  };

  var memberlist = (
    <div>
      <Member name="Nguyễn Hữu Trung" age="42" />
      <Member name="Hữu Trung" age="40" />
    </div>
  );

  ReactDOM.render(
    memberlist,
    document.getElementById('member')
  );
</script>
```

Với cách khai báo một biến mới **memberlist**, ta có thể sử dụng nhiều component cùng lúc, việc này sẽ giúp giải quyết nhiều cấu trúc sau này.

React.js Props

- Tái sử dụng Component ở nhiều nơi khác nhau: Component member với thông tin “Hữu Trung", “40" tuổi, tuy nhiên thông tin này được sử dụng ở một khu vực khác, không thuộc về id="member", thì ta chỉ việc thêm một render mới:

```
<div id="member"></div>
<div id="client"></div>

<script type="text/babel">
    function Member(props){
    return(
      <div className="member">
        <h2>{ props.name }</h2>
        <p>Tuổi: { props.age }</p>
      </div>
    )
  };
  ReactDOM.render(
    <Member name="Nguyễn Hữu Trung" age="42" />,
    document.getElementById('member')
  );
  ReactDOM.render(
    <Member name="Hữu Trung" age="40" />,
    document.getElementById('client')
  );
</script>
```

- Khi này Nguyễn Hữu Trung sẽ thuộc về **id="member"**, Hữu Trung sẽ thuộc về **id="client"**.

React.js props xử lý data

- Bài học này sẽ giúp bạn xử lý dữ liệu truyền vào cho một component

```
<div id="member"></div>
<script type="text/babel">
    function Member(props){
    return(
      <div className="member">
        <h2>{ props.name }</h2>
        <p>Tuổi: { props.age }</p>
      </div>
    )
  };
  ReactDOM.render(
    <Member name="Nguyễn Hữu Trung" age="42" />,
    document.getElementById('member')
  );
</script>
```

- Thay vì viết dữ liệu trực tiếp bên trong phần render, ta sẽ tách riêng phần dữ liệu ra một dữ liệu của một đối tượng (data Object).
- Khi này ta cần xử lý lại phần render sao cho có thể lấy được dữ liệu, thao tác rất đơn giản như sau:

```
<div id="member"></div>
<script type="text/babel">
    function Member(props){
    return(
      <div className="member">
        <h2>{ props.name }</h2>
        <p>Tuổi: { props.age }</p>
      </div>
    )
  };
  const info = {
    name: 'Nguyễn Hữu Trung',
    age: '42'
  };

  ReactDOM.render(
    <Member name={info.name} age={info.age} />,
    document.getElementById('member')
  );
</script>
```

- **const info** khai báo giá trị cho một object.
- **info.name** lấy giá trị name của object.
- **info.age** lấy giá trị age của object

React.js component xử lý data

- Giúp các bạn hình dung được các dữ liệu được truyền giữa các component như thế nào.
- Tạo một data object như sau

```
const member = {
  text: 'Giảng viên khoa CNTT - HCMUTE',
  info: {
  path: '3x4_trung.jpg',
  name: 'Nguyễn Hữu Trung',
  age: 42
  }
}
```

- Lần lượt tiến hành xử lý dữ liệu trong các function như sau

```
function Avatar(props){
  return(
  <div className="avatar">
    <img src={props.info.path} alt="" />
  </div>
  )
};
```

```
function MemberInfo(props){
  return(
  <div className="info">
    <h2>{props.info.name}</h2>
    <p>Tuổi: {props.info.age}</p>
  </div>
  )
};
```

```
function Comment(props){
  return(
  <div className="comment">
    {props.text}
  </div>
  )
};
```

```
function Member(){
    return(
      <div className="member">
        <Avatar info={member.info} />
        <MemberInfo info={member.info} />
        <Comment text={member.text} />
      </div>
    )
  };
  ReactDOM.render(
    <Member />,
    document.getElementById('member')
  );
```

- **{props.info.path}** props sẽ lấy giá trị path từ info của dữ liệu, tương tự cho name và age.
- **{props.text}** giá trị text được lấy trực tiếp.
- **<Avatar info={member.info} />** cần khai báo các giá trị dữ liệu tương ứng, tương tự cho function **<MemberInfo />** và **<Comment />.**

React.js component xử lý data

- Xử lý dữ liệu khi các component lồng vào nhau (tái sử dụng component)
- Để hiểu rõ hơn việc truyền dữ liệu giữa các component, ta tiến hành lồng function Avatar(props) vào bên trong MemberInfo(props), ta làm như sau (nhớ xóa bỏ function Avatar(props) bên trong Member()):
  - Việc di chuyển <Avatar info={member.info} /> từ function Member() vào bên trong MemberInfo(props). Khi này kết quả vẫn không thay đổi.
  - Thay vì sử dụng member.info của function <Avatar />, ta sẽ liên kết <Avatar /> với <MemberInfor /> bằng một props.avatar, với avatar là một tên bất kỳ được sử dụng riêng bên trong function <Avatar />.
  - avatar ta có thể đặt tên bất kỳ, miễn sao ở function <MemberInfor /> ta gọi kết quả trả về là props.info là được. Điều này tiện lợi trong việc xử lý các giá trị props ở các component mà không cần quan tâm tới các giá trị props ở các component khác .

```
function Avatar(props){
 return(
   <div className="avatar">
     <img src={props.avatar.path} alt="" />
   </div>
 )
};
function MemberInfo(props){
 return(
   <div className="info">
     <Avatar avatar={props.memberInfo} />
     <h2>{props.memberInfo.name}</h2>
     <p>Tuổi: {props.memberInfo.age}</p>
   </div>
 )
};
```

```
function Member(){
 return(
   <div className="member">
     <MemberInfo memberInfo={member.info} />
     <Comment text={member.text} />
   </div>
 )
};
function Comment(props){
 return(
   <div className="comment">
     {props.text}
   </div>
 )
};
```

React.js component xử lý data

- Xử lý dữ liệu tại phần render:
  - Hiện tại giá trị dữ liệu object **const member** được gọi trực tiếp bên trong **Member(),** ta có thể gọi giá trị này tại phần render, mục đích để tách biệt rõ các component chỉ nên xử lý các thao tác logic, phần dữ liệu nên đặt tại render.
  - Ta viết lại component Member() và phần render như sau

```
function Member(props){
  return(
    <div className="member">
      <MemberInfo memberInfo={props.info} />
      <Comment text={props.text} />
    </div>
  )
};
ReactDOM.render(
  <Member info={member.info} text={member.text} />,
  document.getElementById('member')
);
```

- Giá trị của Props **không thay đổi**, chỉ truyền từ component này sang component khác, giá trị dữ liệu phụ thuộc vào dữ liệu object khai báo ban đầu.

Props - arrow function

- Hàm mũi tên (arrow function) đã được giới thiệu trong bài ES6 - arrow function, cấu trúc có những vượt trội hơn so với cách sử dụng cũ, sử dụng tham số linh hoạt hơn

Cấu trúc function thông thường	Cấu trúc arrow function

`function Member(props){`
`  return(`
`  <div className="member">`
`  <h2>{ props.name }</h2>`
`  <p>Tuổi: { props.age }</p>`
`  </div>`
`  )`
`};`
	`var Member = (props) => {`
	`  return(`
	`  <div className="member">`
	`  <h2>{ props.name }</h2>`
	`  <p>Tuổi: { props.age }</p>`
	`  </div>`
	`  )`
	`};`

- Nhìn vào so sánh cấu trúc bên trên, ta thấy cấu trúc bên trong function không có gì thay đổi, chỉ thay đổi cách khai báo function.

Props - arrow function

- Ví dụ:

```
let Avatar = (props) =>{
 return(
   <div className="avatar">
     <img src={props.avatar.path} alt="" />
   </div>
 )
};
var MemberInfo = (props) =>{
 return(
   <div className="info">
     <Avatar avatar={props.memberInfo} />
     <h2>{props.memberInfo.name}</h2>
     <p>Tuổi: {props.memberInfo.age}</p>
   </div>
 )
};
```

```
let Comment = (props) =>{
 return(
   <div className="comment">
     {props.text}
   </div>
 )
};
var Member = (props) => {
 return(
   <div className="member">
     <MemberInfo memberInfo={props.info} />
     <Comment text={props.text} />
   </div>
 )
};
```

```
ReactDOM.render(
  <Member info={member.info} text={member.text} />,
  document.getElementById('member')
);
```

React.js Props by Class

- Trong phiên bảng Javascript ES6, chúng ta có thể chuyển đổi một function sang dạng Class, việc này có nhiều ưu điểm hơn so với cách viết function, thao tác với các object khá tốt, các bạn có thể tìm hiểu thêm tại ES6 - classes.

Cấu trúc Function	Cấu trúc Class
`function Member(props){ return( <div className="member"> <h2>{ props.name }</h2> <p>Tuổi: { props.age }</p> </div> ) };`	`class Member extends React.Component { render() { return( <div className="member"> <h2>{ this.props.name }</h2> <p>Tuổi: { this.props.age }</p> </div> ) } };`

- Nhìn vào so sánh cấu trúc bên trên, ta thấy dễ dàng chuyển đổi một function sang Class:
  - function Member(props) thành class Member extends React.Component.
  - Thêm render() {}.
  - Ở mỗi props thêm this.

React.js Props by Class

- Ví dụ:

```
<div id="member"></div>
<script type="text/babel">
  const member = {
      text: 'Giảng viên khoa CNTT - HCMUTE',
      info: {
      path: '3x4_trung.jpg',
      name: 'Nguyễn Hữu Trung',
      age: 42
      }
    }
   var Avatar = (props) =>{
    return(
      <div className="avatar">
        <img src={props.avatar.path} alt="" />
      </div>
    )
   };
  var MemberInfo = (props) =>{
    return(
      <div className="info">
        <Avatar avatar={props.memberInfo} />
        <h2>{props.memberInfo.name}</h2>
        <p>Tuổi: {props.memberInfo.age}</p>
      </div>
    )
  };
```

```
class Comment extends React.Component {
  render(){
      return(
      <div className="comment">
        {this.props.text}
      </div>
      )
  }
};
class Member extends React.Component {
  render() {
    return(
      <div className="member">
        <MemberInfo memberInfo={this.props.info} />
        <Comment text={this.props.text} />
      </div>
    )
  }
};

ReactDOM.render(
  <Member info={member.info} text={member.text} />,
  document.getElementById('member')
);
</script>
```

React.js State

- Giá trị của Props không thay đổi, chỉ truyền từ component này sang component khác.
- Tuy nhiên trong thực tế, bản thân component đôi lúc cũng cần xử lý dữ liệu riêng của nó, ví dụ thay đổi thông tin dữ liệu như name, ... do đó React đã cho ra đời khái niệm State để giải quyết bài toán về dữ liệu bên trong component.
- Cấu trúc chung của một component sử dụng với State được viết dưới dạng Classes (hoặc arrow function) như bên dưới

```
class ClassName extends React.Component {
  constructor(props) {
    super(props);
    this.state = {};
  }

  render() {
    return (
      ...
    );
  }
}
```

- **ClassName** tên Class.
- **extends React.Component** phương thức kế thừa component (thực ra là kế thừa class đã tồn tại).
- **constructor(props)** hàm khởi tạo đối tượng cho một class, mỗi class chỉ chứa một hàm khởi tạo duy nhất.
- **super(props)** gọi lại constructor trong **React.Component**, khi này ta mới có thể sử dụng phương thức this được.
- **this.state** phải là một Object.
- **render** giống như cách dùng của props.

React.js State


```
<div id="member"></div>

<script type="text/babel">
  class Member extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          avatar:"3x4_trung.jpg",
          name: "Nguyễn Hữu Trung",
          age: 42
        };
      };
      render() {
        return (
          <div>
            <img src={this.state.avatar} alt="" />
            <h2>{this.state.name}</h2>
            <p>Tuổi: {this.state.age}</p>
          </div>
        );
      };
    };

    ReactDOM.render(
      <Member />,
      document.getElementById('member')
    );
  </script>
```

Ví dụ:

Nguyễn Hữu Trung

Tuổi: 42

React.js setState

- Muốn thay đổi hay cập nhật nội dung của một State, chúng ta sử dụng this.setState khi này dữ liệu thay đổi sẽ được cập nhật tới State hiện tại

```
<div id="member"></div>
<script type="text/babel">
  class Member extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          avatar:"3x4_trung.jpg",
          name: "Nguyễn Hữu Trung",
          age: 42
        };
      };
      changeName = () => {
        this.setState({
          avatar:"3x4_trung.jpg",
          name: "Hữu Trung",
          age: 40
        })
      };
      render() {
        return (
          <div>
            <img src={this.state.avatar} alt="" />
            <h2>{this.state.name}</h2>
            <p>Tuổi: {this.state.age}</p>
             <button type="button" onClick={this.changeName}>Change name</button>
          </div>
        );
      };
    };
    ReactDOM.render(
      <Member />,
      document.getElementById('member')
    );
  </script>
```

React.js setState

- Ta sẽ thử sử dụng setState với dữ liệu Object nhiều phần tử để biết thêm nhiều trường hợp khác nhau

```
<div id="member"></div>
<script type="text/babel">
  class Member extends React.Component {
    constructor(props) {
      super(props);
      this.state = {
        users: [
        { name: "Nguyễn Hữu Trung", age: 42, avatar:"3x4_trung.jpg" }
        ],
        job: "Giảng viên"
      };
    };
    changeName = () => {
      this.setState({
        users: [
        { name: "Hữu Trung", age: 40, avatar:"1.jpg" }
        ],
        job: "Giảng viên khoa CNTT - HCMUTE"
      })
    };
    render() {
      return (
        <div>
          <img src= {this.state.users[0].avatar} width="150" height="150" />
          <h2>{this.state.users[0].name}</h2>
          <p>Tuổi: {this.state.users[0].age}</p>
          <p>Nghề nghiệp: {this.state.job}</p>
           <button type="button" onClick={this.changeName}>Change name</button>
        </div>
      );
    };
  };
  ReactDOM.render(
    <Member />,
    document.getElementById('member')
```

Component Lifecycle

- Component có thể được viết theo kiểu của vòng đời này như sau:
  - Tạo vòng đời (Mount trên DOM)
  - Cập nhật vòng đời (Updating)
  - Kết thúc vòng đời (Unmount từ DOM)
- Quá trình trên được gọi là Component lifecycle (vòng đời của một component).

Component Lifecycle

Lifecycle	Câu lệnh	Mô tả
Mount	componentWillMount()	Phương thức này sẽ được gọi trước khi render. Không nên sử dụng `setState()` tại đây vì chưa có DOM để tương tác.
	componentDidMount()	Phương thức này sẽ được gọi một lần duy nhất sau khi component được render xong, thường sử dụng `setState()` vì khi này DOM đã được tạo, thường được dùng khi muốn: ▪ Get data từ server. ▪ Lấy giá trị cố định của thành phần như: width, height, offset, ... ▪ Khai báo setInterval cho hành động lặp lại. ▪ Khai báo sự kiện như: load, scroll, resize, ... ▪ ...
Updating	shouldComponentUpdate()	Phương thức này xác định component nên được update hay không, phương thức này có 2 giá trị: ▪ `true` (mặc định) khi này sẽ tiếp tục thực hiện `componentWillUpdate()` và `componentDidUpdate()`. ▪ `false` sẽ dừng thực hiện `componentWillUpdate()` và `componentDidUpdate()`.
	componentWillUpdate()	Phương thức này được gọi một lần duy nhất sau `shouldComponentUpdate()`, sử dụng trước khi re-render component.
	componentDidUpdate()	Chúng ta gọi phương thức này sau khi DOM đã được update xong, tại đây thường được dùng khi muốn: ▪ Xử lý dữ liệu được lấy từ server. ▪ Thay đổi giao diện dựa vào dữ liệu nhận được. ▪ ...
Unmount	componentWillUnMount()	Phương thức này sẽ được gọi một lần duy nhất trước khi component được loại bỏ từ DOM, những thứ đã khai báo tại `componentDidMount()` cần phải được hủy. Đây cũng là phương thức kết thúc vòng đời của component.

Component Lifecycle

- Ví dụ: Ta tạo một vòng đời (lifecycle) cho một bộ đếm (counter) tăng từ 0 cho đến 3, khi đạt đến giá trị 3 thì sẽ xóa bộ đếm để kết thúc vòng đời, ta lần lượt thực hiện các bước sau:
  - Khai báo ban đầu, render giá trị đầu tiên.
  - Cho giá trị bắt đầu tăng sau mỗi 1 giây (cập nhật State).
  - Loại bỏ component ra khỏi DOM khi giá trị đạt đến 3 (Xóa component để xem kết thúc vòng đời).

Component Lifecycle

- Tạo function count để cập nhật giá trị với setState.
- Cần setInterval để giá trị tự động cập nhật function count.
- Do component đã được render xong, nên setInterval ta đặt bên
  trong componentDidMount.

```
<div id="counter"></div>
<script type="text/babel">
  class Counter extends React.Component {
    constructor(props) {
      super(props);
      this.state = { count: 1 };
    };
    countFnc = () => {
      this.setState({
      count: this.state.count + 1
      });
    };
    componentDidMount() {
      setInterval(() => this.countFnc(), 1000);
      console.log("componentDidMount");
    };
    render() {
      console.log('count = ' + this.state.count);
      return (
      <h2>{this.state.count}</h2>
      );
    };
  };
  ReactDOM.render(
    <Counter />,
    document.getElementById('counter')
  );
  </script>
```

Component Lifecycle

- Tận dụng việc gọi một lần duy nhất của phương thức **componentDidUpdate** để loại bỏ component ra khỏi DOM khi giá trị đạt đến 3.
- Sử dụng **ReactDOM.unmountComponentAtNode()** để loại bỏ component ra khỏi DOM.
- Sau khi Component đã được loại bỏ khỏi DOM, ta cần sử dụng phương thức **componentWillUnmount()** để xóa interval đã đăng ký từ **componentDidMount(),** nếu không sẽ báo lỗi vì vòng đời không được kết thúc.

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 1 };
  };
  countFnc = () => {
    this.setState({
      count: this.state.count + 1
    });
  };
  componentDidMount() {
    this.counterID = setInterval(() => this.countFnc(), 1000);
    console.log("componentDidMount");
  };
  componentDidUpdate(prevProps, prevState) {
    console.log("componentDidUpdate");
    if (this.state.count === 3) {
      ReactDOM.unmountComponentAtNode(document.getElementById('counter'));
    }
  };
  componentWillUnmount() {
    console.log("componentWillUnmount");
    clearInterval(this.counterID);
  };

  render() {
    console.log('count = ' + this.state.count);
    return (
      <h2>{this.state.count}</h2>
    );
  };
};
```

React.js xử lý sự kiện (event)

- Cách React.js xử lý sự kiện (event) (thật ra là JSX) có khác biệt so với viết HTML thông thường, xem so sánh sau:

Event viết theo HTML	Event viết theo JSX

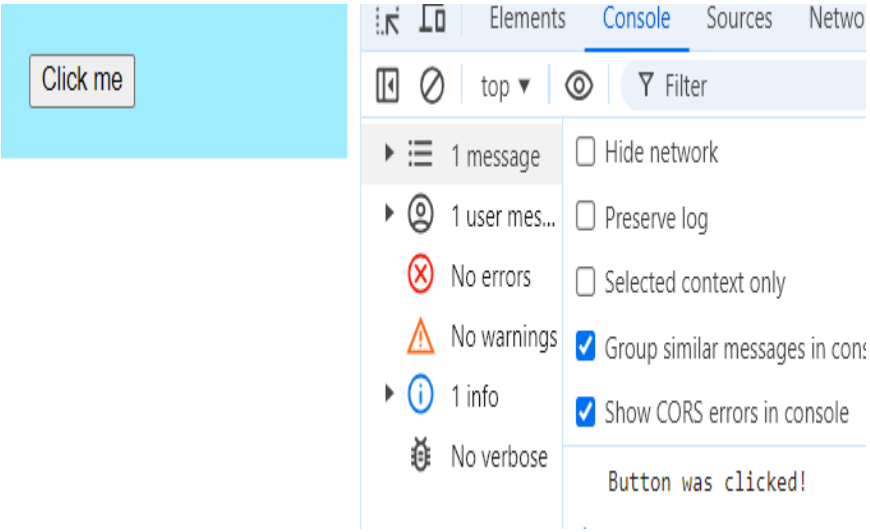
`<button onclick="fncName()">`
`  Click me!`
`</button>`
	`<button onClick={fncName}>`
	`  Click me!`
	`</button>`

- Thay đổi bên trên khác biệt ở phần onClick={} thay vì onclick="", (cẩn thận chữ C viết hoa).
- Thay đổi khác là ở phần gọi function:
  - Khi bạn khai báo fncName() thì function sẽ được gọi khi render.
  - Khi bạn khai báo fncName thì function sẽ được gọi khi button được click

React.js xử lý sự kiện (event)

- React event với <button>

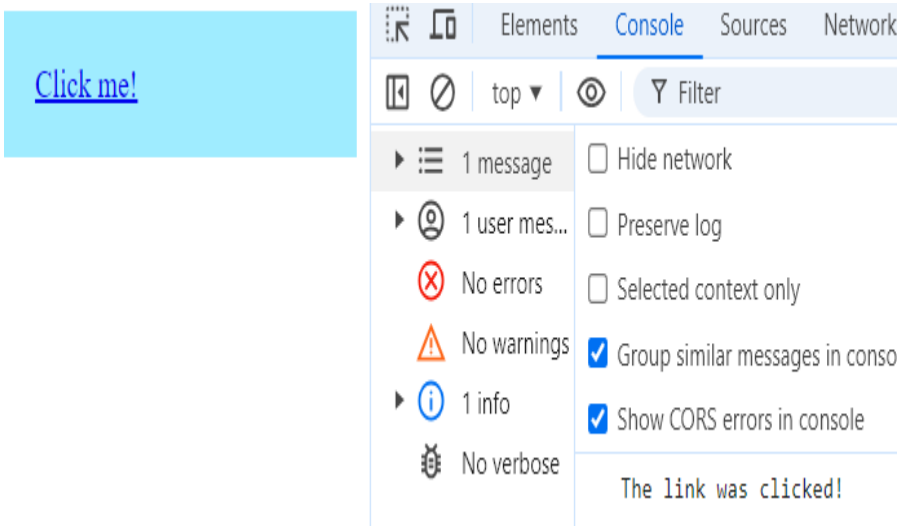
```
<div id="member"></div>

<script type="text/babel">
  const ActionButton = () => {
      const handleClick = () => {
        console.log('Button was clicked!');
      }

      return (
        <button onClick={handleClick}>
          Click me
        </button>
      );
    }

    ReactDOM.render(
      <ActionButton />,
      document.getElementById('member')
    );
  </script>
```



React.js xử lý sự kiện (event)

- React event với liên kết **<a>**
  - Đối với liên kết **<a>** thì sử dụng return false để ngăn mặc định của click, mà bắt buộc gọi **e.preventDefault()** trong function

```
<div id="member"></div>

<script type="text/babel">
  const ActionLink = () => {
    const handleClick = (e) => {
      e.preventDefault();
      console.log('The link was clicked!');
    };

    return (
      <a href="#" onClick={handleClick}>
        Click me!
      </a>
    );
  };

  ReactDOM.render(
    <ActionLink />,
    document.getElementById('member')
  );
</script>
```



React.js form

- Ta có thể sử dụng các thành phần của form trong trang HTML, tuy nhiên để tận dụng ưu điểm của React, cũng như có thể xử lý dễ dàng với dữ liệu, React chọn cách viết các thành phần bên trong file Javascript, kỹ thuật này được gọi là "controlled component".
- Các giá trị thay đổi trong form sẽ được giữ trong State của component, và được cập nhật thay đổi thông qua setState.

```
<div id="root"></div>
<script type="text/babel">
  class ActionButton extends React.Component {
    constructor(props) {
      super(props);
      this.state = {value: ''};
    };
    handleChange = (event) => {
      this.setState({value: event.target.value});
    };
    handleSubmit = (event) => {
      alert('Giá trị đã được submit: ' + this.state.value);
      event.preventDefault();
    };
    render() {
      return (
        <form onSubmit={this.handleSubmit}>
          <label>
            Name:
            <input type="text" value={this.state.value} onChange={this.handleChange} />
          </label>
          <input type="submit" value="Submit" />
        </form>
      );
    };
  };
  ReactDOM.render(
    <ActionButton />,
    document.getElementById('root')
  );
</script>
```

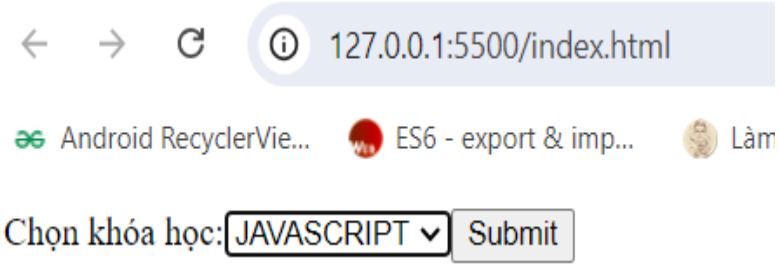
React.js form


```
<div id="root"></div>
<script type="text/babel">
  class ActionButton extends React.Component {
      constructor(props) {
        super(props);
        this.state = {value: 'Your message here'};
      };
      handleChange = (event) => {
        this.setState({value: event.target.value});
      };
      handleSubmit = (event) => {
        alert('Giá trị đã được submit: ' + this.state.value);
        event.preventDefault();
      };
      render() {
        return (
          <form onSubmit={this.handleSubmit}>
            <label>
              Message:
              <textarea type="text" value={this.state.value} onChange={this.handleChange} />
            </label>
            <input type="submit" value="Submit" />
          </form>
        );
      };
    };
    ReactDOM.render(
      <ActionButton />,
      document.getElementById('root')
    );
  </script>
```

React.js form


```
<div id="root"></div>
<script type="text/babel">
  class ActionButton extends React.Component {
      constructor(props) {
        super(props);
        this.state = {value: 'javascript'};
      };
      handleChange = (event) => {
        this.setState({value: event.target.value});
      };
      handleSubmit = (event) => {
        alert('Giá trị đã được submit: ' + this.state.value);
        event.preventDefault();
      };
      render() {
        return (
          <form onSubmit={this.handleSubmit}>
            <label>
              Chọn khóa học:
              <select value={this.state.value} onChange={this.handleChange}>
                <option value="html">HTML</option>
                <option value="css">CSS</option>
                <option value="javascript">JAVASCRIPT</option>
                <option value="react.js">REACT.JS</option>
              </select>
            </label>
            <input type="submit" value="Submit" />
          </form>
        );
      };
    };
    ReactDOM.render(
      <ActionButton />,
      document.getElementById('root')
```

- Cách xử lý cho **<select><option>** có một điểm khác biệt hơn so với xử lý bên file HTML, đó là ta sẽ sử dụng thuộc tính value ngay tại <select>, việc này sẽ tiện lợi ở chỗ: **State** chỉ cần tương tác với một value của <select>.



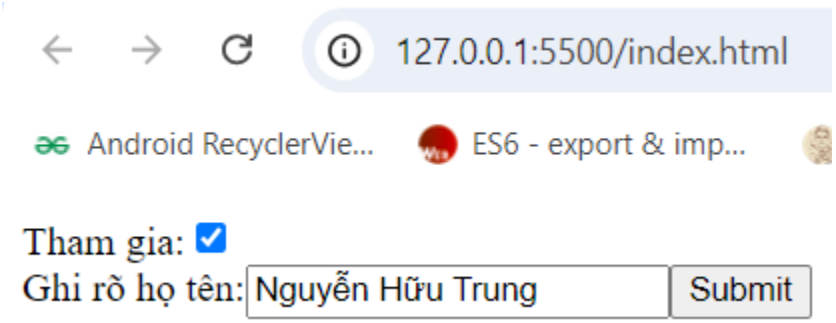
React.js form


```
<div id="root"></div>
<script type="text/babel">
  class ActionButton extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          isGoing: true,
          guestName: "Nguyễn Hữu Trung"
        };
      };
      handleInputChange = (event) => {
        const target = event.target;
        const value = target.type === 'checkbox' ? target.checked : target.value;
        const name = target.name;
        this.setState({
          [name]: value
        });
      }
      handleSubmit = (event) => {
        alert('Giá trị đã được submit: '
          + this.state.isGoing + ' và '
          + this.state.guestName
        );
        event.preventDefault();
      };
```

React.js form


```
    render() {
      return (
        <form onSubmit={this.handleSubmit}>
          <label>
            Tham gia:
            <input
              name="isGoing"
              type="checkbox"
              checked={this.state.isGoing}
              onChange={this.handleInputChange} />
          </label>
          <br />

          <label>
            Ghi rõ họ tên:
            <input
              name="guestName"
              type="text"
              value={this.state.guestName}
              onChange={this.handleInputChange} />
          </label>
          <input type="submit" value="Submit" />
        </form>
      );
    };
  };
  ReactDOM.render(
    <ActionButton />,
    document.getElementById('root')
  );
</script>
```

- Ở phần constructor khai báo 2 giá trị đầu tiên của <input type="checkbox" /> và <input type="text" />.
- Trong function handleInputChange xử lý giá trị true và false của <input type="checkbox" /> đồng thời lấy giá trị name của <input type="text" />.
- Cũng trong function handleInputChange sử dụng setState để thực hiện cập nhật giá trị mới cho State.
- Funciton handleSubmit giữ vai trò gọi 2 giá trị mới của <input type="checkbox" /> và <input type="text" />.



React.js form kết hợp nhiều thành phần


```
<div id="root"></div>
<script type="text/babel">
  class ActionButton extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          isGoing: true,
          guestName: 'Nguyễn Hữu Trung',
          course: 'react.js',
          message: 'Giảng viên Khoa CNTT-HCMUTE'
        };
      };
      handleInputChange = (event) => {
        const target = event.target;
        const value = target.type === 'checkbox' ? target.checked : target.value;
        const name = target.name;

        this.setState({
          [name]: value
        });
      }
      handleSubmit = (event) => {
        alert('Tham gia: '
          + this.state.isGoing + ', họ tên: '
          + this.state.guestName + ', '
          + this.state.course + ' và '
          + this.state.message
        );
        event.preventDefault();
      };
```

React.js form kết hợp nhiều thành phần


```
render() {
  return (
    <form onSubmit={this.handleSubmit}>
      <label>
        Tham gia:
        <input
          name="isGoing"
          type="checkbox"
          checked={this.state.isGoing}
          onChange={this.handleInputChange} />
      </label>
      <br />
      <label>
        Ghi rõ họ tên:
        <input
          name="guestName"
          type="text"
          value={this.state.guestName}
          onChange={this.handleInputChange} />
      </label>
      <br />
```

```
      <label>
        Chọn khóa học:
        <select
          name="course"
          value={this.state.course}
          onChange={this.handleInputChange}
        >
          <option value="html">HTML</option>
          <option value="css">CSS</option>
          <option value="javascript">JAVASCRIPT</option>
          <option value="react.js">REACT.JS</option>
        </select>
      </label>
      <br />

      <label>
        Message:
        <textarea
          name="message"
          type="text"
          value={this.state.message}
          onChange={this.handleInputChange}
        />
      </label>
      <br />

      <input type="submit" value="Submit" />
    </form>
```

```
        );
      };
    };
    ReactDOM.render(
      <ActionButton />,
      document.getElementById('root')
    );
</script>
```

Tham gia: ☑
Ghi rõ họ tên: Nguyễn Hữu Trung
Chọn khóa học: REACT.JS
Message: Giảng viên Khoa CNTT-HCMUTE
Submit

React.js render có điều kiện

- Có thể render bất kỳ component nào ta muốn, bằng cách sử dụng điều kiện **if else** tại phần render. React sẽ dựa vào câu điều kiện để tạo thành phần đại diện cho State hiện tại, sau đó sẽ dựa vào setState để cập nhập lại giao diện.
- Chúng ta sẽ tạo một ứng dụng sao cho: Nếu chưa login (hoặc logout) thì sẽ hiển thị "Please sign in". Nếu đã login thì sẽ hiển thị "Welcome back!".
- Ta lần lượt thực hiện các bước sau:
  - Tạo 2 component UserGreeting (hiển thị nội dung khi login) và GuestGreeting (hiển thị nội dung khi chưa login hoặc logout)
  - Tạo 1 function Greeting điều khiển 2 component User và Guess tuy theo điều kiện user có login hay không.
  - Tạo 2 component hiện thị nội dung cho button: LogoutButton khi đã login và LoginButton khi chưa login hoặc logout.
  - Tạo 2 function handleLoginClick và handleLogoutClick để xử lý click button.
  - Việc còn lại cuối cùng là tiến hành xử lý câu điều kiện bên trong phần render để trả đúng trạng thái user có login hay không

React.js render có điều kiện

- **Component UserGreeting và GuestGreeting**

```
const UserGreeting = (props) => {
  return <h2>Welcome back!</h2>;
}
```

```
const GuestGreeting = (props) => {
  return <h2>Please sign in.</h2>;
}
```

- **Function Greeting:** Nội dung function này lọc điều kiện login:
  - Nếu login thì trả về component UserGreeting
  - Nếu chưa login hoặc logout thì trả về component GuestGreeting..

```
const Greeting = (props) => {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return
      <UserGreeting />;
  }
  return
    <GuestGreeting />;
}
```

React.js render có điều kiện

- **Component LoginButton và LogoutButton:** dùng để điều khiển hiển thị của button: nếu là login thì button sẽ hiển thị Logout và ngược lại.

```
const LoginButton = (props) => {
  return (
    <button onClick={props.onClick}>
      Login
    </button>
  );
}
```

```
const LogoutButton = (props) => {
  return (
    <button onClick={props.onClick}>
      Logout
    </button>
  );
}
```

- **function handleLoginClick và handleLogoutClick:** sử dụng setState để cập nhật trạng thái login hay logout.

```
handleLoginClick = () => {
  this.setState({isLoggedIn: true});
}
```

```
handleLogoutClick = () => {
  this.setState({isLoggedIn: false});
}
```

React.js render có điều kiện

- Kết quả

```
<div id="root"></div>
<script type="text/babel">
  class LoginControl extends React.Component {
    constructor(props) {
      super(props);
      this.state = {isLoggedIn: false};
    };
    handleLoginClick = () => {
      this.setState({isLoggedIn: true});
    };
    handleLogoutClick = () => {
      this.setState({isLoggedIn: false});
    };
    render() {
      const isLoggedIn = this.state.isLoggedIn;
      let button; /* Khai báo biến button sử dụng cho if */

      if (isLoggedIn) {
        button = <LogoutButton onClick={this.handleLogoutClick} />;
      } else {
        button = <LoginButton onClick={this.handleLoginClick} />;
      }
      return (
        <div>
          <Greeting isLoggedIn={isLoggedIn} />
          {button}
        </div>
      );
    };
  };
  const UserGreeting = (props) => {
    return <h2>Welcome back!</h2>;
  };
  const GuestGreeting = (props) => {
    return <h2>Please sign up.</h2>;
  };
  const Greeting = (props) => {
    const isLoggedIn = props.isLoggedIn;
    if (isLoggedIn) {
      return <UserGreeting />;
    }
    return <GuestGreeting />;
  };
  const LoginButton = (props) => {
    return (
      <button onClick={props.onClick}>
        Login
      </button>
    );
  };
  const LogoutButton = (props) => {
    return (
      <button onClick={props.onClick}>
        Logout
      </button>
    );
  };
  ReactDOM.render(
    <LoginControl />,
    document.getElementById('root')
  );
</script>
```

React.js render có điều kiện

- Render có điều kiện bên trong cấu trúc JSX
- Render có điều kiện có thể được viết bên trong một cấu trúc JSX, như sau:

```
render() {
  const isLoggedIn = this.state.isLoggedIn;
  return (
  <div>
      The user is <b>{isLoggedIn ? 'currently' : 'not'}</b> logged in.
  </div>
);
}
```

- Hoặc được viết như sau

```
render() {
  const isLoggedIn = this.state.isLoggedIn;
  return (
    <div>
      {isLoggedIn ? (
      <LogoutButton onClick={this.handleLogoutClick} />
      ) : (
      <LoginButton onClick={this.handleLoginClick} />
      )}
    </div>
  );
}
```

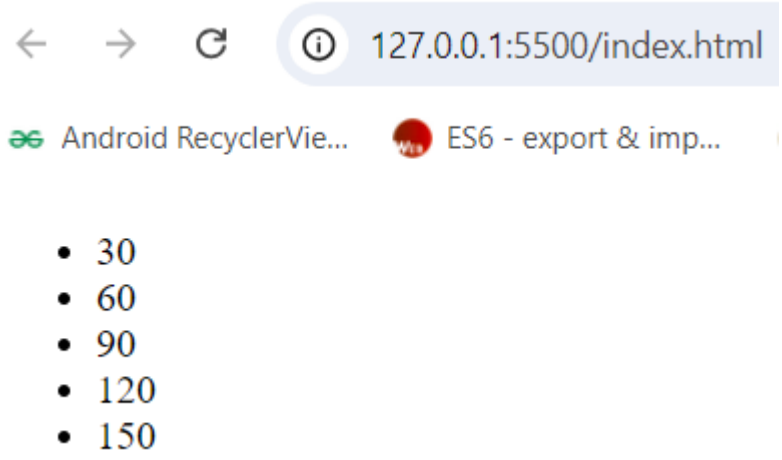
React.js Array to list

- Dữ liệu dưới dạng Array, Object làm sao có thể chuyển đổi sang cấu trúc là một danh sách (list) của HTML
- Render giá trị Array: Để lấy giá trị là một Array, đối với Javascript ta làm như sau

```
const numbers = [10, 20, 30, 40, 50];
const doubled = numbers.map((number) => number * 3);
console.log(doubled);
```

- Tương tự như vậy, để render giá trị Array ra một list ta làm như sau

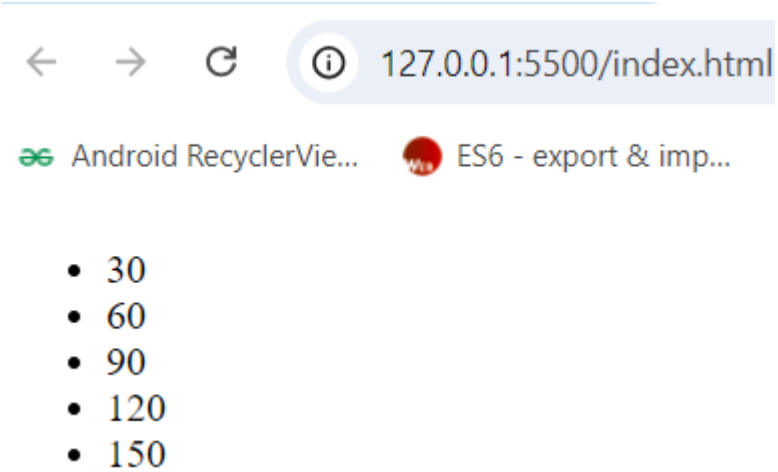
```
const numbers = [10, 20, 30, 40, 50];
const listItems = numbers.map((number) =>
  <li>{number * 3}</li>
);
ReactDOM.render(
  <ul>{listItems}</ul>,
  document.getElementById('root')
);
```



React.js Array to list

- Render giá trị Array viết dạng component

```
<div id="root"></div>
<script type="text/babel">
  function NumberList(props) {
    const numbers = props.numbers;
    const listItems = numbers.map((number) =>
    <li>{number * 3}</li>
    );
    return (
    <ul>{listItems}</ul>
    );
  }
  const numbers = [10, 20, 30, 40, 50];
  ReactDOM.render(
  <NumberList numbers={numbers} />,
  document.getElementById('root')
  );
</script>
```



- Ta thấy kết quả vẫn hiển thị như mong muốn, tuy nhiên lúc này khi xem output console (Nhấn F12), thì sẽ thấy thông báo Warning: Each child in a list should have a unique "key" prop..
- Thuộc tính key cần được thêm khi tạo list.

React.js list và keys

- **key** giúp React xác định item nào được thay đổi, được thêm, hoặc được xóa.
- **key** dành cho mỗi phần tử trong list, do đó keys của mỗi phần tử phải là duy nhất.

```
<div id="root"></div>
<script type="text/babel">
  function NumberList(props) {
    const numbers = props.numbers;
    const listItems = numbers.map((number) =>
    <li key={number.toString()}>
      {number * 3}
    </li>
    );
    return (
      <ul>{listItems}</ul>
    );
  };
  const numbers = [10, 20, 30, 40, 50];
  ReactDOM.render(
  <NumberList numbers={numbers} />,
  document.getElementById('root')
  );
</script>
```

- Xem lại kết quả console khi này, ta sẽ thấy không còn cảnh báo nữa.
- Do **key** mỗi phần tử là duy nhất, nên **key** thường là giá trị **id**, trường hợp không có id hoặc giá trị nào đó duy nhất thì ta có thể sử dụng index như là giải pháp cuối cùng.

```
const todoItems = todos.map((todo) =>
  <li key={todo.id}>
    {todo.text}
  </li>
);
const todoItems = todos.map((todo, index) =>
  <li key={index}>
    {todo.text}
  </li>
);
```

React.js list và keys

- Trích xuất component với Keys: Chú ý là key chỉ có ý nghĩa trong ngữ cảnh có các mảng đang sử dụng, ví dụ như nếu trích xuất component **ListItem**, thì ta nên gán key ngay tại **<ListItem key={} />** thay vì tại <li> bên trong function ListItem.

Cấu trúc không đúng

```
<div id="root"></div>
<script type="text/babel">
  function ListItem(props) {
    const value = props.value;
    return (
      /* Sai: không cần key ở đây */
      <li key={value.toString()}>
        {value}
      </li>
    );
  };
  function NumberList(props) {
    const numbers = props.numbers;
    const listItems = numbers.map((number) =>
      /* Sai: key nên được sử dụng ở đây: */
      <ListItem value={number * 3} />
    );
    return (
      <ul>
        {listItems}
      </ul>
    );
  };
  const numbers = [10, 20, 30, 40, 50];
  ReactDOM.render(
    <NumberList numbers={numbers} />,
    document.getElementById('root')
  );
</script>
```

Cấu trúc đúng

```
<div id="root"></div>
<script type="text/babel">
  function ListItem(props) {
    return <li>{props.value}</li>;
  };
  function NumberList(props) {
    const numbers = props.numbers;
    const listItems = numbers.map((number) =>
      /* Sử dụng đúng */
      <ListItem key={number.toString()} value={number * 3} />
    );
    return (
      <ul>
        {listItems}
      </ul>
    );
  };

  const numbers = [10, 20, 30, 40, 50];
  ReactDOM.render(
    <NumberList numbers={numbers} />,
    document.getElementById('root')
  );
</script>
```

React.js list và keys

- Keys phải giống nhau trong cùng một component: Chú ý là key cần phải đồng nhất khi sử dụng trong cùng một component (hoặc trong cùng một function hay class).

```
<div id="root"></div>
<script type="text/babel">
  function Tutorial(props) {
    const sidebar = (
      <ul>
        {props.posts.map((post) =>
          <li key={post.id}>
            {post.title}
          </li>
        )}
      </ul>
    );
    const content = props.posts.map((post) =>
      <div key={post.id}>
        <h2>{post.title}</h2>
        <p>{post.content}</p>
      </div>
    );
    return (
      <div>
        {sidebar}
        <hr />
        {content}
      </div>
    );
  };
  const posts = [
    {id: 1, title: 'Học Javascript', content: 'Chào mừng bạn đến Javascript!'},
    {id: 2, title: 'Học React.js', content: 'Bạn có thể bắt đầu học React.js sau khi cài đặt.'}
  ];
  ReactDOM.render(
    <Tutorial posts={posts} />,
    document.getElementById('root')
  );
```

- Chú ý là key không phải là một giá trị của component, do đó nếu muốn sử dụng giá trị giống như key thì ta cần phải tạo một props, sử dụng với tên khác.

```
const content = posts.map((post) =>
  <Post key={post.id} id={post.id} title={post.title} />
);
```

React.js nhúng map() vào JSX

- Thay vì khai báo một map() tách riêng phần render, ta hoàn toàn có thể sử dụng bên trong cấu trúc JSX của phần render.

map() bên ngoài render

```
function ListItem(props) {
  return <li>{props.value}</li>;
};

function NumberList(props) {
  const numbers = props.numbers;
  const listItems = numbers.map((number) =>
    <ListItem key={number.toString()}
              value={number * 2} />
  );
  return (
    <ul>
      {listItems}
    </ul>
  );
};
```

map() bên trong JSX của render

```
function ListItem(props) {
  return <li>{props.value}</li>;
};

function NumberList(props) {
  const numbers = props.numbers;
  return (
    <ul>
      {numbers.map((number) =>
        <ListItem key={number.toString()}
                  value={number * 2} />
      )}
    </ul>
  );
};
```

- Ta thấy JSX cho phép chúng ta nhúng một biểu thức bên trong cặp dấu {}, việc này giúp code rõ ràng hơn.

React.js state với map()

- Áp dụng map() vào một state thực tế, mục đích giúp các bạn hình dung rõ hơn việc ứng dụng map() để list ra một danh sách từ dữ liệu ban đầu.

```
<div id="member"></div>
<script type="text/babel">
  class Member extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          users: [
            { id: 1, name: "Nguyễn Hữu Trung", age: 42 },
            { id: 2, name: "Hữu Trung", age: 43 },
            { id: 3, name: "Trung Nguyễn", age: 22 }
          ],
          job: "Giảng viên khoa CNTT-HCMUTE"
        };
      };
       render() {
        return (
        <div>
          <ul>
            {this.state.users.map(item => (
            <li key={item.id}>
              <h2>Tên: {item.name}</h2>
              <p>Tuổi: {item.age}</p>
            </li>
            ))}
          </ul>
        </div>
        );
       };
    };
    ReactDOM.render(
      <Member />,
      document.getElementById('member')
    );
  </script>
```

React.js setState với map()

- Áp dụng map() vào một setState() để cập nhật một mục bất kỳ trong một mảng cho trước. Để cập nhật nội dung, ta cần một <button> và một function xử lý nội dung cập nhật, ta thêm nội dung trên như sau:

```
changeItem = () => {
 let key = 2;

 this.setState(prevState => ({
   users: prevState.users.map(
     item => item.id === key? {
       ...item,
       name: 'Trung Nguyễn Hữu',
       age: 41
     }: item
   )
 }));
};
render() {
 return (
 <div>
   <ul>
     {this.state.users.map(item => (
     <li key={item.id}>
       <h2>Tên: {item.name}</h2>
       <p>Tuổi: {item.age}</p>
     </li>
     ))}
   </ul>
    <button type="button" onClick={this.changeItem}>Update</button>
 </div>
 );
};
```

- **let key = 2;**, thực tế trong việc update, khi click vào liên kết update sẽ trả về giá trị id của mục cần được update, key này sẽ đại diện cho id đó.
- **item.id === key**, nếu item.id bằng với key thì tiến hành cập nhật nội dung, không thì trả về item.

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

React.js project component

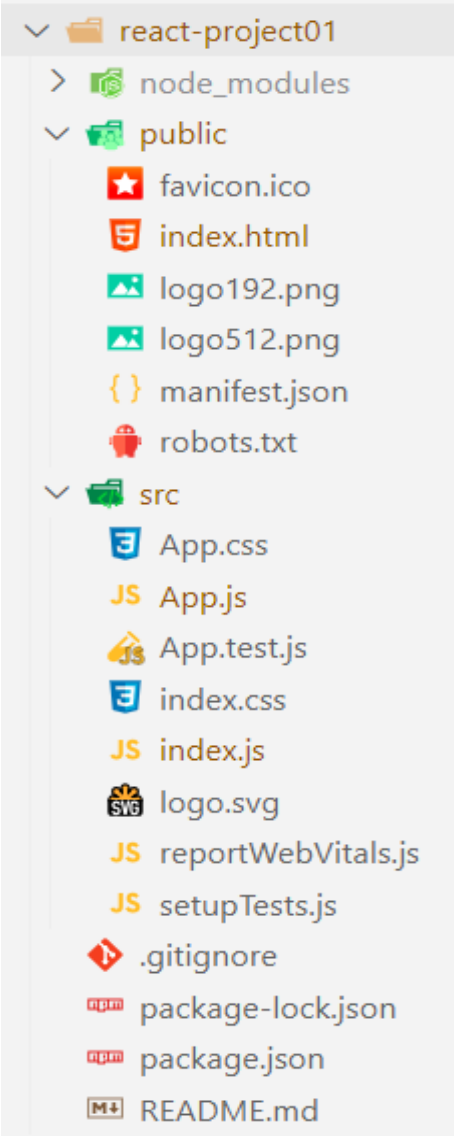
THS. NGUYỄN HỮU TRUNG

Nội dung

- Nội dung chuẩn bị:
  - Cài đặt để chạy được project React.js, xem lại phần cài đặt React.js
  - Template mẫu, tải tại đây
- Ta sẽ thực hiện các bước sau:
  - Đưa template mẫu vào project React.
  - Phân tích các component trong template mẫu.
  - Tách template mẫu ra thành từng component.
  - Kết nối các component lại với nhau.

Đưa template mẫu vào project React

- Sau khi cài đặt xong React ta sẽ có project **react-project01** với cấu trúc thư mục như sau



- Trong template mẫu, có 2 file chính, là **index.html** và **/css/style.css**, nhiệm vụ của ta là làm sao cho nội dung của template mẫu này hiển thị thông qua project React.
- Để làm được việc đó, ta lần lượt thực hiện như sau:
  - Copy nội dung code bên trong style.css của template thay thế code bên trong **/src/App.css.**
  - Copy nội dung phần của template thay thế cho nội dung bên trong phần return của **/src/App.js**.

Đưa template mẫu vào project React

App.css M ×

react-project01 > src > App.css > ...

```
/* Reset lại một vài thuộc tính */
* {
    margin: 0;
    padding: 0;
}
ol, ul { list-style: none; }

img { border: none; vertical-align: top; }

/* Nội dung layout */
#layout {
    margin: 20px auto;
    width: 1024px;
}
header, footer, #content, aside, .box-list li {
    font-size: 30px;
    text-align: center;
}
header {
    background-color: #92dfc8;
    height: 80px;
    line-height: 80px;
}
main {
    display: flex;
    flex-wrap: wrap;
}
#content {
    background-color: #caeff2;
    height: 400px;
    line-height: 400px;
    width: 774px;
}
aside {
    background-color: #f2d8ca;
    height: 400px;
    line-height: 400px;
    width: 250px;
}
.box-list {
    display: flex;
    margin-top: 10px;
}
.box-list li {
    background-color: #f2f1ca;
    height: 150px;
    line-height: 150px;
    text-align: center;
    margin-right: 10px;
    margin-bottom: 10px;
    width: 248.5px;
}
.box-list li:last-child {
    margin-right: 0;
}
footer {
    background-color: #eecbf3;
    height: 80px;
    line-height: 80px;
}
```

Đưa template mẫu vào project React

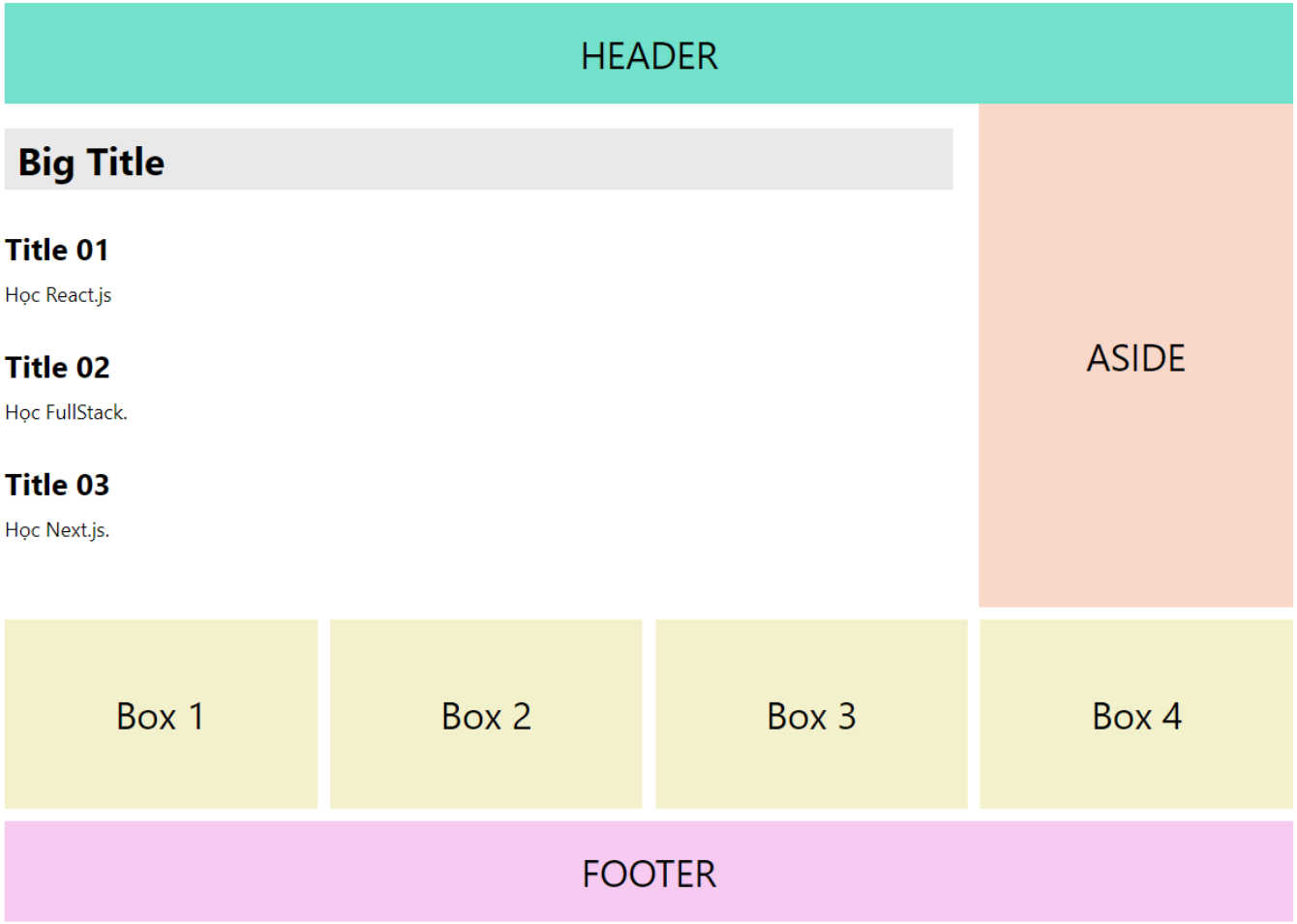
JS App.js M ×

react-project01 > src > JS App.js > ...

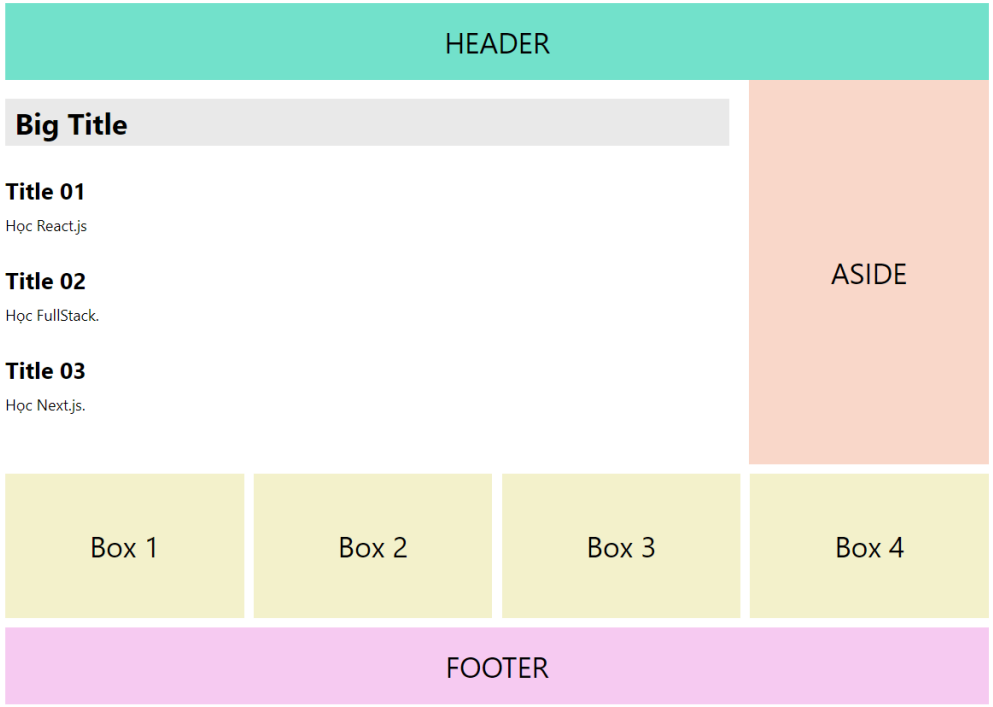
```
1  import React from 'react';
2  import './App.css';
3  function App() {
4    return (
5      <div id="layout">
6        <header>HEADER</header>
7        <main>
8          <section id="content">
9            <h1>Big Title</h1>
10           <ul className="item-list">
11             <li>
12               <h2>Title 01</h2>
13               <div>
14                 Học React.js
15               </div>
16             </li>
17             <li>
18               <h2>Title 02</h2>
19               <div>
20                 Học FullStack.
21               </div>
22             </li>
23             <li>
24               <h2>Title 03</h2>
25               <div>
26                 Học Next.js.
27               </div>
28             </li>
29           </ul>
30         </section>
31         <aside>
32           ASIDE
33         </aside>
34         <ul className="box-list">
35           <li>Box 1</li>
36           <li>Box 2</li>
37           <li>Box 3</li>
38           <li>Box 4</li>
39         </ul>
40       </main>
41       <footer>FOOTER</footer>
42     </div>
43   );
44 };
45 export default App;
```

Đưa template mẫu vào project React

- Chỉ cần thay thế 2 file như trên thôi, là ta đã đưa nội dung của template mẫu vào project React thành công.
- Để chạy project, ta dùng lệnh **npm start** sẽ thấy được kết quả sau



Phân tích component



- Theo đó ta lần lượt có các component như sau:
  - **Header**: là component chứa nội dung của phần header.
  - **Headline**: là component chứa nội dung của "Big Title".
  - **ItemList**: là component chứa nội dung của nhóm "Title 01" cho tới "Title 03".
  - **Aside**: là component chứa nội dung của phần sidebar.
  - **BoxList**: là component chứa nội dung danh sách "BOX".
  - **Footer**: là component chứa nội dung của phần footer.

Tách template mẫu ra thành từng component.

- Để tiện quản lý về sau, ta sẽ tiến hành tách nội dung của file /src/App.js ra từng component tương ứng.
- Trước tiên ta sẽ tạo folder components đặt bên trong thư mục /src/, tạo các file Javascript với tên tương ứng với từng component, và nội dung là nội dung được tách từ file /src/App.js, ta có cấu trúc thư mục như sau:

JS Header.js U ×

react-project01 > src > components > JS Hea

```
1  import React from 'react';
2
3  const Header = () => {
4    return (
5      <header>HEADER</header>
6    )
7  }
8
9  export default Header;
```

JS Headline.js U ●

react-project01 > src > components > JS Hea

```
1  import React from 'react';
2
3  const Headline = () => {
4    return (
5      <h1>Big Title</h1>
6    )
7  }
8
9  export default Headline;
```

JS Aside.js U ×

react-project01 > src > components > JS Aside

```
1  import React from 'react';
2
3  const Aside = () => {
4    return (
5      <aside>
6        ASIDE
7      </aside>
8    )
9  }
10
11 export default Aside;
```

Tách template mẫu ra thành từng component.

JS ItemList.js U ×

react-project01 > src > components > JS ItemList.js > .

```js
import React from 'react';
const ItemList = () => {
  return (
      <ul className="item-list">
            <li>
              <h2>Title 01</h2>
              <div>
                Học React.js
              </div>
            </li>
            <li>
              <h2>Title 02</h2>
              <div>
                Học FullStack.
              </div>
            </li>
            <li>
              <h2>Title 03</h2>
              <div>
                Học Next.js.
              </div>
            </li>
          </ul>
  )
}
export default ItemList;
```

JS BoxList.js U ×

react-project01 > src > components > JS BoxLis

```js
import React from 'react';

const BoxList = () => {
  return (
    <ul className="box-list">
      <li>Box 1</li>
      <li>Box 2</li>
      <li>Box 3</li>
      <li>Box 4</li>
    </ul>
  )
}

export default BoxList;
```

JS Footer.js U ×

react-project01 > src > components > JS Foo

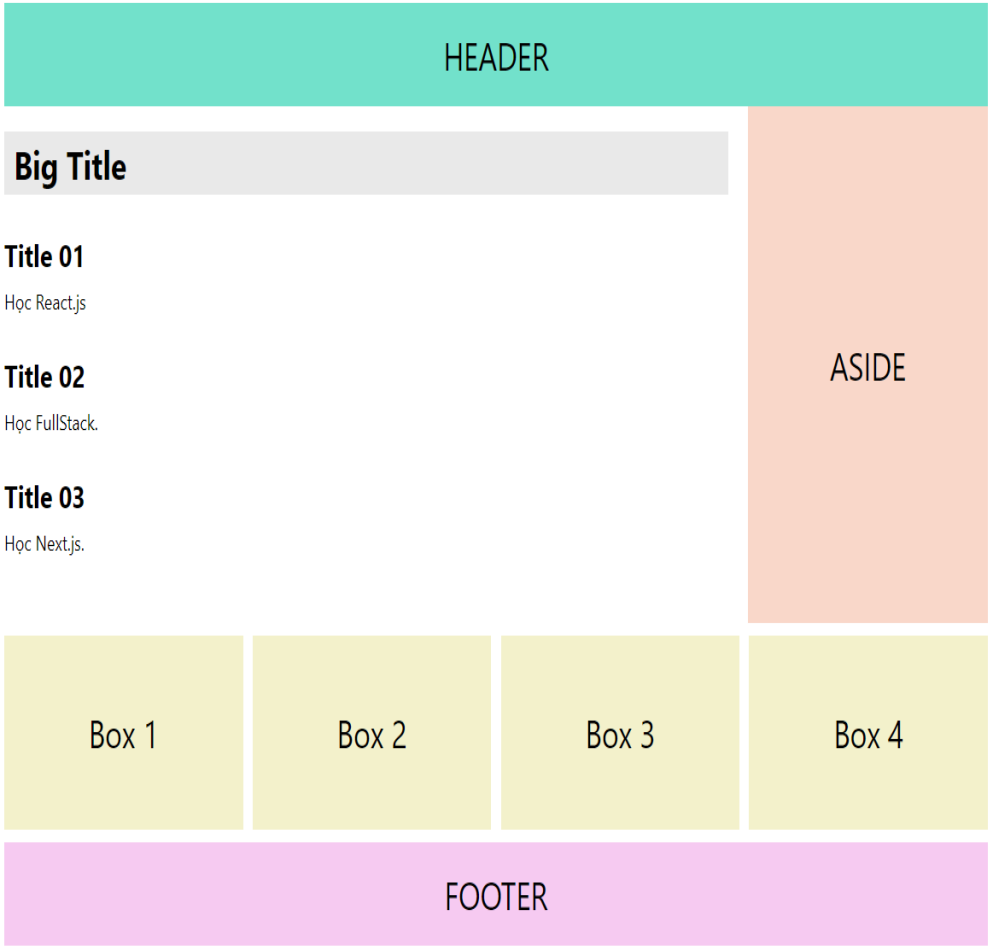
```js
import React from 'react';

const Footer = () => {
  return (
    <footer>FOOTER</footer>
  )
}

export default Footer;
```

Kết nối các component lại với nhau

JS App.js M ×

react-project01 > src > JS App.js > App > render

```
1  import React, { Component } from 'react';
2  import './App.css';
3  import Header from './components/Header';
4  import Headline from './components/Headline';
5  import ItemList from './components/ItemList';
6  import Aside from './components/Aside';
7  import BoxList from './components/BoxList';
8  import Footer from './components/Footer';
9
10 class App extends Component {
11   render() {
12     return (
13       <div id="layout">
14         <Header />
15         <main>
16           <section id="content">
17             <Headline />
18             <ItemList />
19           </section>
20           <Aside />
21           <BoxList />
22         </main>
23         <Footer />
24       </div>
25     );
26   }
27 }
28 export default App;
```



Tạo data cần cho các component


```
28  //tạo data cho components
29  const blog = {
30    bigTitle: 'Hướng dẫn học React.js.',
31    itemList: [
32      {
33        title: 'Giới thiệu',
34        info: 'React.js là gì.'
35      },
36      {
37        title: 'Component của React.js',
38        info: 'Thành phần quan trọng của React.js là component.'
39      },
40      {
41        title: 'Props and State',
42        info: 'Quản lý trạng thái và lấy dữ liệu.'
43      }
44    ],
45    boxList: [
46      'Nguyễn Hữu Trung',
47      'React.js',
48      'NodeJS',
49      'Express.js'
50    ]
51  };
```

- Ta đặt tên dữ liệu là blog, các dữ liệu thay đổi sẽ là bigTitle, itemList và boxList của các component tương ứng, đồng thời ta thay đổi một số dữ liệu so với file template mẫu, để cho dễ biết dữ liệu đã được hiển thị.

Tạo data cần cho các component

JS App.js M ×

react-project01 > src > JS App.js > App > render

```
1  import React, { Component } from 'react';
2  import './App.css';
3  import Header from './components/Header';
4  import Headline from './components/Headline';
5  import ItemList from './components/ItemList';
6  import Aside from './components/Aside';
7  import BoxList from './components/BoxList';
8  import Footer from './components/Footer';
9
10 class App extends Component {
11   render() {
12     return (
13       <div id="layout">
14         <Header />
15         <main>
16           <section id="content">
17             <Headline bigTitle={blog.bigTitle} />
18             <ItemList />
19           </section>
20           <Aside />
21           <BoxList />
22         </main>
23         <Footer />
24       </div>
25     );
26   }
27 };
```

JS Headline.js U ×

react-project01 > src > components > JS Headlin

```
1  import React from 'react';
2
3  const Headline = (props) => {
4    return (
5      <h1>{props.bigTitle}</h1>
6    )
7  }
8  export default Headline;
```

Xử lý data trong component ItemList

JS ItemList.js U ×

react-project01 > src > components > JS ItemList.js > Ite

```
1   import React from 'react';
2
3   const ListItem = (props) => {
4     return (
5       <li>
6         <h2>{props.propsItem.title}</h2>
7         <div>
8           {props.propsItem.info}
9         </div>
10      </li>
11    )
12  };
13
14  const ItemList = (props) => {
15    const propsList = props.itemList;
16
17    return (
18      <ul className="item-list">
19        {propsList.map((item) =>
20          <ListItem propsItem={item} />
21        )}
22      </ul>
23    )
24  };
25
26  export default ItemList;
```

App.js

```
<ItemList itemList={blog.itemList} />
```

- Do dữ liệu của thuộc tính itemList có 2 cấp, nên cách xử lý khác hơn so với xử lý component Headline.
- Nội dung phần này sẽ áp dụng cách xử lý dữ liệu giống như bài React.js - nhúng map() vào JSX

Xử lý data trong component BoxList

JS BoxList.js U ×

react-project01 > src > components > JS BoxList.js > default

```js
import React from 'react';

const BoxItem = (props) => {
  return (
    <li>
      {props.propsBox}
    </li>
  )
};

const BoxList = (props) => {
  const boxList = props.boxList;

  return (
    <ul className="box-list">
      {boxList.map((itemBox) =>
        <BoxItem propsBox={itemBox} />
      )}
    </ul>
  )
};

export default BoxList;
```

App.js

```js
<BoxList boxList={blog.boxList} />
```

Next.js	React.js	NodeJS	Express.js

Viết lại App.js theo state

JS App.js M ×

react-project01 > src > JS App.js > App > render

```
1  import React, { Component } from 'react';
2  import './App.css';
3  import Header from './components/Header';
4  import Headline from './components/Headline';
5  import ItemList from './components/ItemList';
6  import Aside from './components/Aside';
7  import BoxList from './components/BoxList';
8  import Footer from './components/Footer';
9
10 class App extends Component {
11   constructor(props){
12     super(props);
13     this.state = blog;
14   };
15   render() {
16     return (
17       <div id="layout">
18         <Header />
19
20         <main>
21           <section id="content">
22             <Headline bigTitle={this.state.bigTitle} />
23             <ItemList itemList={this.state.itemList} />
24           </section>
25
26           <Aside />
27           <BoxList boxList={this.state.boxList} />
28         </main>
29         <Footer />
30       </div>
31     );
32   };
33 };
```

- **this.state = blog** chúng ta liên kết dữ liệu với state thông qua **this.state**.
- Khi này chúng ta có thể sử dụng **this.state** để gọi giá trị dữ liệu của **blog**.

Xử lý thay đổi dữ liệu trong component

- Sử dụng **setState** thay đổi dữ liệu của **ItemList**.
- Trước tiên ta cần tạo **button** để thực hiện hành động thay đổi, và tạo dữ liệu thay đổi bên trong setState, ta tiến hành như sau

App.js

```
<section id="content">
  <Headline bigTitle={this.state.bigTitle} />
  <ItemList itemList={this.state.itemList} />
  <button type="button" onClick={this.changeItem}>Change Data</button>
</section>
```

```
//dữ liệu sẽ thay đổi
changeItem = () => {
    this.setState({
      itemList: [
        {
          title: 'Giới thiệu React.js',
          info: 'Khái niệm React.js.'
        },
        {
          title: 'Thành phần của React.js',
          info: 'Components là thành phần quan trọng của React.js.'
        },
        {
          title: 'Cập nhật dữ liệu bằng setState',
          info: 'Props and State.'
        }
      ]
    });
};
```

HEADER

Hướng dẫn học React.js.

Giới thiệu React.js

Khái niệm React.js.

Thành phần của React.js

Components là thành phần quan trọng của React.js.

Cập nhật dữ liệu bằng setState

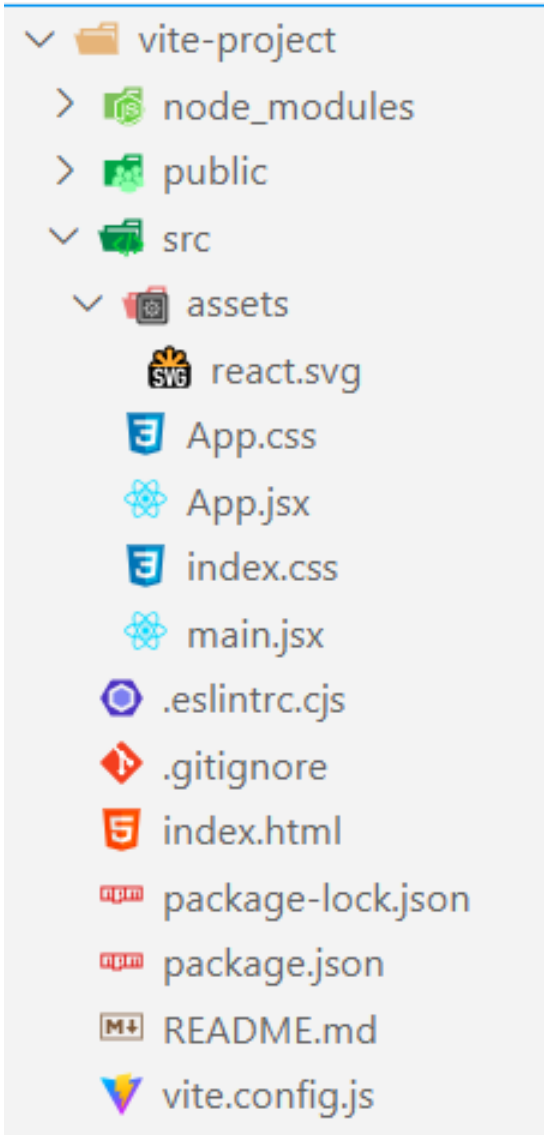
Props and State.

Change Data

Cài đặt React.js với thư viện build vitejs

- **ViteJS** là một công cụ mã nguồn mở (build tool) cho việc phát triển ứng dụng web. ViteJS sinh ra để hỗ trợ việc dev và build nhanh hơn.
- **ViteJS** được phát triển bởi **Evan You**, người sáng tạo và đã làm việc trên dự án **Vue.js**.
- Cài đặt ViteJS và sử dụng nó trong một dự án React:
  - Cài đặt Node.js và npm trên máy tính của bạn.
  - Sử dụng npm để cài đặt ViteJS bằng lệnh sau: **npm create vite@latest**
  - Đặt tên project: **vite-project**
    - Chọn React để cài React và React-dom và chọn Javascript + JWC hoặc TypeScript + JWC
  - Chuyển vào thư mục project: **cd vite-project**
  - Cài đặt module: **npm install**
  - Chạy project: **npm run dev**
  - Biên dịch và deploy thì dùng lệnh build**: npm run build**

Cài đặt React.js với thư viện build vitejs



package.json

vite-project > package.json > ...

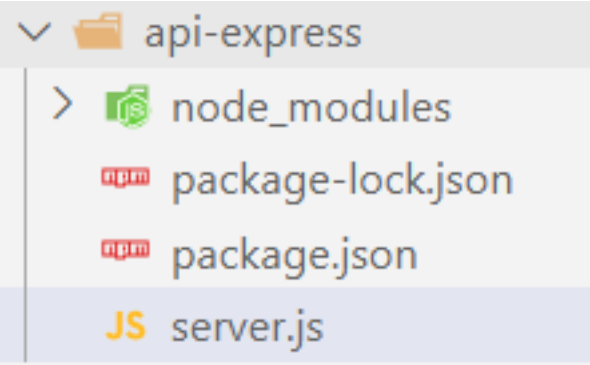
```
{
  "name": "vite-project",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react-swc": "^3.5.0",
    "eslint": "^8.57.0",
    "eslint-plugin-react": "^7.34.3",
    "eslint-plugin-react-hooks": "^4.6.2",
    "eslint-plugin-react-refresh": "^0.4.7",
    "vite": "^5.3.4"
  }
}
```

http://localhost:5173/

React.js api

- Khi làm việc với React, chúng ta sẽ thường có nhu cầu liên kết dữ liệu như MySQL, MongoDB, ..., tuy nhiên bản thân React là một Javascript, chỉ thao tác ở phía client, việc liên kết sử lý dữ liệu là không thể. Để giải quyết vấn đề này, chúng ta sẽ liên kết dữ liệu với React thông qua API
- Bước 1: Tạo project API bằng Express.js
  - Tạo thư mục: **api-express**
  - Chuyển vào thư mục**: cd api-express**
  - Tạo file package.json: **npm init -y**
  - Cài express: **npm install --save express** và **npm i --save-dev @types/express**
  - Chạy: **node server.js**
- Bước 2: Tạo và gọi API

Bước 1



package.json

api-express > package.json > ...

```
1  {
2    "name": "api-express",
3    "version": "1.0.0",
4    "main": "server.js",
     Debug
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "express": "^4.19.2"
14   },
15   "devDependencies": {
16     "@types/express": "^4.17.21"
17   },
18   "type": "module"
19 }
```

Bước 1

JS server.js X

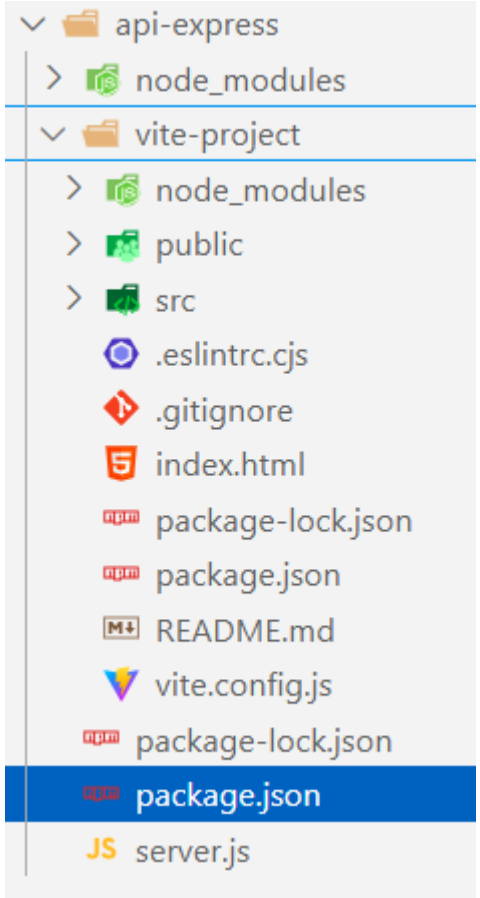
api-express > JS server.js > ...

```
1  import express from 'express'; //sử dụng cho type:module
2  const app = express();
3
4  app.get('/', (req, res) => res.send('Nguyễn Hữu Trung!'));
5
6  (async () => {
7      try {
8          //using mongoose
9          // await connection();
10         app.listen(4000, () => {
11             console.log(`Backend Nodejs App listening on port ${4000}`)
12         })
13     } catch (error) {
14         console.log(">>> Error connect to DB: ", error)
15     }
16 })()
```

- express = require('express') hoặc import express from 'express'; //sử dụng cho type:module , gọi thư viện Express.
- app.get('/', (req, res) => res.send('Nguyễn Hữu Trung')); chính là một API đơn giản gọi nội dung Hello World.
- app.listen(4000, () =>) dòng này quy định port cho ứng dụng.

React.js api

- Bước 2:
  - Cài đặt React với vitejs (xem lại phần trước)
  - Cài đặt **concurrently** để liên kết cổng server và client
    - Vào thư mục api-express gõ **npm install –save concurrently**
    - Chỉnh lại file package.json (ở slide tiếp theo)
    - Chạy project: **npm run dev**



React.js api

package.json

api-express > package.json > {} scripts > dev

```
1  {
2    "name": "api-express",
3    "version": "1.0.0",
4    "main": "server.js",
     Debug
5    "scripts": {
6      "server": "node server.js",
7      "client": "npm run dev --prefix vite-project",
8      "dev": "concurrently \"npm run server\" \"npm run client\""
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "description": "",
14   "dependencies": {
15     "concurrently": "^8.2.2",
16     "express": "^4.19.2"
17   },
18   "devDependencies": {
19     "@types/express": "^4.17.21"
20   },
21   "type": "module"
22 }
```

package.json

api-express > vite-project > package.json > ...

```
1  {
2    "name": "vite-project",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
     Debug
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0",
10     "preview": "vite preview"
11   },
12   "dependencies": {
13     "react": "^18.3.1",
14     "react-dom": "^18.3.1"
15   },
16   "devDependencies": {
17     "@types/react": "^18.3.3",
18     "@types/react-dom": "^18.3.0",
19     "@vitejs/plugin-react-swc": "^3.5.0",
20     "eslint": "^8.57.0",
21     "eslint-plugin-react": "^7.34.3",
22     "eslint-plugin-react-hooks": "^4.6.2",
23     "eslint-plugin-react-refresh": "^0.4.7",
24     "vite": "^5.3.4"
25   },
26   "proxy": "http://localhost:4000"
27 }
```

- "**server**": "node server.js" server chạy bằng lệnh này.
- "**client**": "npm run start --prefix **vite-project**" client chạy bằng lệnh này.
- "**dev**": "concurrently \"npm run server\" \"npm run client\"" chạy kết hợp giữa server và client bằng lệnh này.
- Chỉnh lại file /client/package.json để khai báo thêm proxy nhận port 4000 từ server:

React.js api

- Lấy dữ liệu từ API bằng dữ liệu JSON, gửi sang project React, tuy nhiên để xử lý các thao tác HTTP REQUEST như các phương thức GET, POST, DELETE,... chúng ta cần cài đặt thư viện Axios để hỗ trợ.
  - Bước 1: Viết API trong server.js

JS server.js

api-express > JS server.js > ...

```
1  import express from 'express';
2  import cors from 'cors';
3
4  const app = express();
5  //config cors
6  app.use(cors());
7
8  //config req.body
9  app.use(express.json()) // for json
10 app.use(express.urlencoded({ extended: true })) // for form data
11
12 app.get('/api/test', (req, res) => {
13   res.json({ message: 'I am a message from Server!'});
14 })
15
16 try {
17     //using mongoose
18     // await connection();
19     app.listen(4000, () => {
20     console.log(`Backend Nodejs App listening on port ${4000}`)
21     })
22 } catch (error) {
23     console.log(">>> Error connect to DB: ", error)
24 }
```

React.js api

- Lấy dữ liệu từ API bằng dữ liệu JSON, gửi sang project React, tuy nhiên để xử lý các thao tác HTTP REQUEST như các phương thức GET, POST, DELETE,... chúng ta cần cài đặt thư viện Axios để hỗ trợ.
  - Bước 2: Cài Axios: **npm install --save axios**
  - Bước 3: Cài cors: **npm install --save cors** và **npm i --save-dev @types/cors**
  - Bước 4:Cấu hình cors trong file **server.js**

```
const cors = require('cors’);
const app = express();
//config cors
app.use(cors());
```

  - Bước 5: Cài **npm install --save react-scripts** và vào file **App.jsx** của project vite-project chỉnh lại

Bước 3: Vào file **App.jsx** của project vite-project chỉnh lại

App.jsx

api-express > vite-project > src > App.jsx > App > render

```jsx
import { Component } from 'react';
import './App.css';
import axios from 'axios';

class App extends Component {
  state = {
    message: ''
  };

  componentDidMount() {
    axios.get('http://localhost:4000/api/test')
        .then(result => this.setState({ message: result.data.message }))
  }

  render() {
    return(
      <div className="App">

        <h1>{ this.state.message }</h1>

      </div>
    )
  }
}

export default App;
```

- import axios from 'axios'; gọi thư viện Axios.
- state = { message: '' }; Khai báo ban đầu cho State giá trị rỗng, để khi lấy được dữ liệu ta sẽ dùng hàm setState để cập nhật lên giá trị mới.
- componentDidMount() các bạn xem lại ở bài Component Lifecycle
- axios.get('/api/test') lấy dữ liệu bằng thư viện Axios và trả về hàm setState để cập nhật giá trị mới cho State.
- this.state.message gọi giá trị State mới được cập nhật.